Benchmarking Rule-Based, Classical Machine Learning, and Deep Learning Anomaly Detectors in ETL Data Quality Pipelines

Theo Bigand
ESILV — M1 Data & Artificial Intelligence
Léonard de Vinci Pôle Universitaire
Courbevoie, France
theobigand@gmail.com

Abstract*—Modern data pipelines increasingly depend on automated data quality controls, yet Extract–Transform–Load (ETL) practitioners still rely mostly on hand-crafted declarative rules. Recent work has started to introduce machine learning (ML) anomaly detectors inside ETL workflows, but systematic, apples-to-apples comparisons between rule-based, classical ML, and deep learning approaches remain scarce—especially when evaluated within a realistic pipeline and broken down by the type of anomaly being injected. This paper presents an open-source ETL pipeline integrating three families of anomaly detectors: statistical rules (Z-score, IQR, Great Expectations), classical ML detectors (Isolation Forest, LOF, One-Class SVM), and deep learning detectors (Autoencoder, LSTM-Autoencoder). We evaluate the three families on three datasets—Kaggle Credit Card Fraud, UCI Water Quality, and a controlled synthetic dataset generated with Faker in which six distinct anomaly types are injected at controlled rates. Beyond aggregate precision, recall, F1, ROC-AUC and PR-AUC, we introduce a per-anomaly-type heatmap that maps each method to the anomaly categories it detects best. Our goal is to give practitioners a reproducible, pipeline-integrated benchmark that answers the practical question: *which family of detectors should I use, for which kind of data quality problem, at what operational cost?

***Index Terms*—data quality, ETL, anomaly detection, benchmarking, machine learning, Great Expectations, Isolation Forest, autoencoder**

I. Introduction

Extract–Transform–Load (ETL) pipelines are the backbone of modern data infrastructure: they move raw operational data into analytical stores where it feeds dashboards, reports, and machine-learning applications [1]. The value delivered downstream is bounded by the quality of the data that reaches the destination, which is why data quality has long been a first-class concern for data engineers. Poor data quality has been estimated to cost organizations an average of $12.9M per year [2], making automated, continuous quality control a critical capability rather than an optional one.

The dominant approach in industry remains rule-based: practitioners hand-write declarative expectations—"column `amount` must be positive", "column `email` must match a regex"—in frameworks such as Great Expectations, Deequ, or Soda [3]. These rules are auditable and cheap to run, but they scale poorly with schema complexity, detect only violations the author anticipated, and miss subtle multi-column or distributional anomalies. To address these limits, a recent line of work has proposed to complement or replace such rules with machine learning anomaly detectors embedded directly inside the ETL flow [2], [4], [5]. The existing literature, however, suffers from three gaps.

First, most published pipelines are conceptual architectures [2], [4] that do not ship with reproducible code or quantitative comparisons. When experiments are reported, they typically exercise a single algorithm on a single dataset—for instance, Isolation Forest on Credit Card Fraud [5], which achieved a near-random ROC-AUC of 0.51 and thus offers limited insight into when ML-based quality control is actually worth adopting.

Second, the anomaly detection community has produced excellent generic benchmarks—ADBench [6] compares 30 algorithms on 57 tabular datasets; PyOD [7] provides a unified implementation of dozens of detectors—but these benchmarks evaluate algorithms *in vacuum*, outside of an ETL context. They do not consider the declarative rule-based baselines that dominate production systems, and they do not measure end-to-end pipeline integration costs (latency, memory, operational overhead).

Third, benchmarks typically report a single aggregated score per dataset, hiding the fact that different families of detectors excel at different kinds of anomalies. A Z-score detector is essentially unable to flag categorical format violations; Great Expectations trivially catches null-rate spikes but ignores multivariate outliers [8]; autoencoders learn joint distributions but can be fooled by concept drift. A practitioner choosing a detector today lacks a fine-grained, actionable map from anomaly type to recommended method.

a) Contributions.: This paper addresses these three gaps. We present:

1) **A reproducible ETL pipeline** integrating rule-based, classical ML, and deep learning anomaly detectors in a

single unified framework, released as open-source code.
2) **A systematic head-to-head benchmark** of three families—statistical rules (Z-score, IQR, Great Expectations), classical ML (Isolation Forest, LOF, One-Class SVM), and deep learning (Autoencoder, LSTM-Autoencoder)—on three datasets (Credit Card Fraud, UCI Water Quality, synthetic Faker), reporting precision, recall, F1, ROC-AUC, PR-AUC, and per-row latency.
3) **A per-anomaly-type analysis.** Using a synthetic dataset in which six distinct anomaly categories (missing values, duplicates, univariate outliers, multivariate outliers, temporal drift, invalid formats) are injected at controlled rates, we produce a method-by-type heatmap that guides the choice of detector in production.

The rest of the paper is organized as follows. Section II reviews related work on data quality in ETL and on anomaly detection benchmarks. Section III describes our pipeline architecture, the detectors we compare, and the datasets and metrics we use. Section IV reports experimental results. Section V discusses when rules, classical ML, and deep learning are each the right tool, and highlights practical trade-offs in latency and operational cost. Section VI concludes and outlines future work.

II. Related Work

To be written. Planned subsections:

- Data quality in ETL pipelines (Khatri & Brown [1]; Data Quality Survey [9])
- Rule-based and declarative approaches (Great Expectations, Deequ, Soda [3])
- Classical ML for anomaly detection (Isolation Forest, LOF, One-Class SVM; Goldstein & Uchida [8]; PyOD [7])
- Deep learning (Autoencoder, LSTM-Autoencoder)
- Existing benchmarks: ADBench [6] and why it does not cover rule-based baselines, ETL integration, or per-anomaly-type analysis
- ML in ETL pipelines: Gopa [2], Marupaka & Rangineni [4], Sokkula & Vuppala [5] as illustrative limitations

III. Methodology

A. Pipeline

Detectors are evaluated inside a minimal Extract–Transform–Load pipeline rather than in isolation. The extract stage reads the raw table, the transform stage removes exact duplicate rows, and the quality stage applies a detector that assigns each row a boolean anomaly flag and a continuous anomaly score. All detectors share a common `fit`/`predict`/`score` interface modelled on PyOD [7], so that classical and deep models can later be dropped in without changing the harness.

B. Rule-based detectors

This stage compares three rule-based baselines. *Z-score* standardizes each numeric column on the fitted data and flags a row when any column exceeds $|z| > 3$. *IQR* learns Tukey's fences $[Q_1 - 1.5\,\mathrm{IQR},\ Q_3 + 1.5\,\mathrm{IQR}]$ per column and flags a row falling outside any fence. *Great Expectations* encodes a declarative expectation suite [3]: a not-null expectation on every column and a range expectation per numeric column, with bounds set to the 0.1th and 99.9th percentiles of the fitted data. A row is flagged when it violates at least one expectation; its score is the fraction of the suite it violates. The continuous scores of the statistical detectors are the worst standardized deviation (Z-score) and the largest fence exceedance in IQR units (IQR).

TABLE I
Rule-based detectors on Credit Card Fraud (full dataset, 284,807 rows). Higher is better except for flagged rows and time. Time is the in-pipeline fit+predict+score latency.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%	Time (s)
Z-score	0.011	0.899	0.022	**0.944**	0.090	13.4	**0.2**
IQR	0.003	**0.968**	0.007	0.922	0.020	48.6	0.6
Great Expectations	**0.039**	0.710	**0.075**	0.848	**0.206**	**3.0**	13.2

C. Dataset and metrics

We report here on the Credit Card Fraud dataset (284,807 transactions, 30 numeric features, 0.17% labelled fraud), a standard and strongly imbalanced benchmark. Each detector is fitted and evaluated on the full feature matrix, and its output is scored against the ground-truth label. Because the detectors emit both a threshold decision and a graded score, we report threshold metrics (precision, recall, F_1) alongside threshold-independent ranking metrics (ROC-AUC and, given the class imbalance, PR-AUC, which is the more informative of the two here). We also report the fraction of rows flagged and wall-clock time. The synthetic per-anomaly-type dataset and the classical/deep detectors are the object of the next stages.

IV. Experiments and Results

Table I reports the three rule-based detectors on Credit Card Fraud. The picture is consistent across metrics: all three recover most frauds (recall 0.71–0.97) but at unusable precision (≤ 0.04), because a fixed statistical rule cannot separate a 0.17% positive class from the long tails of legitimate traffic. IQR is the most aggressive, flagging almost half of all rows, which collapses its PR-AUC. The declarative Great Expectations suite is the most selective (it flags 3% of rows) and obtains the best precision and PR-AUC, while the Z-score gives the best ROC-AUC. No rule-based method reaches a precision that would be acceptable in production, which is the gap the classical and deep detectors introduced in the next stage are meant to close. Measuring detection inside the pipeline also exposes a cost gap: the declarative Great Expectations suite is about $60\times$ slower than the statistical rules ($13.2\,\mathrm{s}$ vs 0.2–$0.6\,\mathrm{s}$ on the full dataset), which matters for its placement in a latency-sensitive ETL flow.

V. Discussion

To be written.

VI. Conclusion

To be written.

References

[1] V. Khatri and C. V. Brown, "Designing data governance," *Communications of the ACM*, vol. 53, no. 1, pp. 148–152, 2010.

[2] R. Gopa, "AI-augmented ETL pipelines for automated data quality, anomaly detection and governance," *International Journal of Computational and Experimental Science and Engineering (IJCESEN)*, vol. 11, no. 4, 2025.

[3] Various Authors, "A survey of pipeline tools for data engineering," *arXiv preprint arXiv:2406.08335*, 2024. [Online]. Available: https://arxiv.org/abs/2406.08335

[4] P. Marupaka and S. Rangineni, "Machine learning-driven predictive data quality assessment in ETL frameworks," *International Journal of Computer Trends and Technology (IJCTT)*, vol. 72, no. 3, 2024.

[5] S. R. Sokkula and S. R. Vuppala, "Implementing machine learning for ETL data transformation and anomaly detection," *International Journal for Multidisciplinary Research (IJFMR)*, vol. 6, no. 6, 2024.

[6] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "ADBench: Anomaly detection benchmark," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://openreview.net/pdf?id=foA_SFQ9zo0

[7] Y. Zhao, Z. Nasrullah, and Z. Li, "PyOD: A python toolbox for scalable outlier detection," *Journal of Machine Learning Research*, vol. 20, no. 96, pp. 1–7, 2019.

[8] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLoS ONE*, vol. 11, no. 4, p. e0152173, 2016.

[9] Various Authors, "A survey on data quality dimensions and tools for machine learning," *arXiv preprint arXiv:2406.19614*, 2024. [Online]. Available: https://arxiv.org/abs/2406.19614